

Simply Typed λ -Calculus

Normal forms, Kripke models, normalisation by evaluation

Andreas Abel

DAT350 / DIT235 *Types for programs and proofs*
Chalmers University of Technology and University of Gothenburg

September 2026

Where we are

Lectures 3 and 4 gave the rules of type theory, one type former at a time.

Lectures 5–7 turned rules into **inductive families**: one constructor per rule, so an induction over derivations is a function defined by pattern matching.

Lecture 6 asked how to show that a formula is *not* derivable, and answered it for the Hilbert system: build a model, prove soundness, exhibit a formula that fails. Peirce's law needed **two stages of knowledge**.

Today: the same questions for the simply typed λ -calculus, that is, for natural deduction in the implicational fragment.

Three notions of derivation, one notion of model, and one algorithm.

Roadmap

First part — syntax

STLC as a mathematical object
Raw terms versus intrinsic typing
Denotational semantics
Peirce's formula resists induction
Normal and neutral derivations
Spines, and no normal Peirce

Second part — semantics

Kripke models, recapitulated
A countermodel for Peirce
Soundness
Weakening
Reification and reflection
Completeness, and normalisation

The Agda development of this lecture is the course source tree `src/stlc`, in particular `Term.agda`, `Term/KripkeModel.agda` and `Term/Normalization/NbE.agda`.

STLC as a mathematical object

The rules

Types, also read as formulas of minimal (implicational) logic:

$$a, b, c ::= \alpha \mid a \Rightarrow b$$

where α ranges over a fixed set of atomic types.

Three rules, for a context Γ of typed assumptions:

$$\frac{x:a \in \Gamma}{\Gamma \vdash x : a} \qquad \frac{\Gamma, x:a \vdash t : b}{\Gamma \vdash \lambda x.t : a \Rightarrow b} \qquad \frac{\Gamma \vdash t : a \Rightarrow b \quad \Gamma \vdash u : a}{\Gamma \vdash tu : b}$$

Curry–Howard: hypothesis, implication introduction, implication elimination. A term *is* a natural deduction derivation.

We write $\Rightarrow$ rather than $\rightarrow$, reserving $\rightarrow$ for Agda's function space — the meta-level.

No library

Everything on the following slides is defined from scratch.

```
module lecture8 where
```

```
data  $\perp$  : Set where
```

```
 $\perp$ -elim : {C : Set}  $\rightarrow \perp \rightarrow C$ 
```

```
 $\perp$ -elim ()
```

$\perp$ is the empty type of Lecture 3. A refutation of a judgment $\mathcal{J}$ is a function $\mathcal{J} \rightarrow \perp$, as in Lectures 5–7.

Types and contexts

```
data BaseTy : Set where  
  X Y : BaseTy
```

```
infixr 9  $\Rightarrow$ 
```

```
data Ty : Set where  
  ' _ : BaseTy  $\rightarrow$  Ty  
   $\Rightarrow$  : Ty  $\rightarrow$  Ty  $\rightarrow$  Ty
```

```
infixl 5  $\cdot$ 
```

```
data Context : Set where  
   $\epsilon$  : Context  
   $\cdot$  : Context  $\rightarrow$  Ty  $\rightarrow$  Context
```

Two decisions in that code

Types are an ordinary inductive type: an atom, or an implication. Two atoms suffice for everything today; nothing but the countermodel of §6 depends on their number.

Contexts are **snoc lists**: the empty context ε , and $\Gamma \cdot a$ for Γ extended on the *right*.

The rules extend a context on the right, and a de Bruijn index counts from the right, so this is the orientation in which no reversal is ever needed.

```
private
variable
   $\alpha$  : BaseTy
   $a\ b\ c$  : Ty
   $\Gamma\ \Delta\ \Phi$  : Context
```

How should terms be represented?

Two designs, both used in practice.

1. Raw terms plus type assignment

- A type Exp of untyped terms.
- A separate judgment $\Gamma \vdash e : a$ as an inductive family.
- Ill-typed terms exist; typing is a relation on them.

2. Intrinsic typing

- One family $\Gamma \vdash a$ of *derivations*.
- Only well-typed terms are representable.
- No separate typing judgment.

Design 1 is the tool of choice when equality of types is non-trivial — for dependent types, where deciding $\Gamma \vdash A = B$ requires evaluation (Lecture 4). It is also what a type checker consumes.

Design 2 is what we use today.

1. Raw terms

```
module Raw where
```

```
open import Agda.Builtin.Nat using (Nat; zero; suc)
```

```
data Exp : Set where
```

```
  var : Nat → Exp
```

```
  abs : Ty → Exp → Exp
```

```
  app : Exp → Exp → Exp
```

```
private
```

```
  variable
```

```
    x : Nat
```

```
    e f : Exp
```

Variables are de Bruijn indices. The implementation in `src/stlc/Exp.agda` uses names instead, and a scope-checking pass; that is an orthogonal concern.

1. Type assignment

```
infix 4 _ $\mapsto$ _  $\in$  _  $\vdash$  _ : _
```

```
data _ $\mapsto$ _  $\in$  _ : Nat  $\rightarrow$  Ty  $\rightarrow$  Context  $\rightarrow$  Set where
```

```
  here :  $\text{zero} \mapsto a \in \Gamma \cdot a$ 
```

```
  there :  $x \mapsto a \in \Gamma \rightarrow \text{suc } x \mapsto a \in \Gamma \cdot b$ 
```

```
data _ $\vdash$ _ : _ (  $\Gamma$  : Context ) : Exp  $\rightarrow$  Ty  $\rightarrow$  Set where
```

```
  var :  $x \mapsto a \in \Gamma \rightarrow \Gamma \vdash \text{var } x : a$ 
```

```
  abs :  $\Gamma \cdot a \vdash e : b \rightarrow \Gamma \vdash \text{abs } a e : a \Rightarrow b$ 
```

```
  app :  $\Gamma \vdash e : a \Rightarrow b \rightarrow \Gamma \vdash f : a \rightarrow \Gamma \vdash \text{app } e f : b$ 
```

One constructor per rule, as in Lectures 5–7. The *term* is an index of the family; the *derivation* is the element.

2. Typed variables

Erase the term index from $_ \mapsto _ \in _$: what remains is a de Bruijn index that carries the type it points at.

infix 4 $_ \in _$

```
data  $\_ \in \_$  (a : Ty) : Context → Set where
  zero :          a ∈ Γ · a
  suc  : a ∈ Γ → a ∈ Γ · b
```

$_ \in _$ is simultaneously

- a proof that a occurs in Γ , and
- a natural number: the position, counted from the right.

The two readings are the same object. This is the pattern of Lecture 5: an index into a structure, and evidence about that structure.

2. Typed terms

infix 4 $_ \vdash _$

```
data  $\_ \vdash \_$  ( $\Gamma$  : Context) : Ty  $\rightarrow$  Set where
  var :  $a \in \Gamma \rightarrow \Gamma \vdash a$ 
  abs :  $\Gamma \cdot a \vdash b \rightarrow \Gamma \vdash a \Rightarrow b$ 
  app :  $\Gamma \vdash a \Rightarrow b \rightarrow \Gamma \vdash a \rightarrow \Gamma \vdash b$ 
```

Read $\Gamma \vdash a$ as *the type of derivations of a from Γ* , and the three constructors as the three rules of §1.

an element of $\Gamma \vdash a$	a well-typed term, or a derivation
pattern matching on it	cases (induction) on the derivation
an ill-typed term	not expressible

Denotational semantics

λ -terms denote functions

STLC $\Gamma \vdash a$ is a deeply embedded domain-specific language for functions. The run function is evaluation of terms into proper functions. (AFP, TDA342)

We interpret each type as a set and each derivation as an element of that set. Fix an interpretation ξ of the atoms:

$$\llbracket \alpha \rrbracket = \xi(\alpha) \qquad \llbracket a \Rightarrow b \rrbracket = \llbracket a \rrbracket \rightarrow \llbracket b \rrbracket$$

A derivation is interpreted in an *environment* that supplies a value for each assumption:

$$\llbracket \Gamma \vdash a \rrbracket : \llbracket \Gamma \rrbracket \rightarrow \llbracket a \rrbracket$$

Implication becomes Agda's function space. Computing the denotation is one form of **evaluation**.

Evaluation in Agda

module Den ($\xi : \text{BaseTy} \rightarrow \text{Set}$) where

Value : Ty $\rightarrow$ Set

Value ($'\alpha$) = $\xi\ \alpha$

Value ($a \Rightarrow b$) = Value $a \rightarrow$ Value b

data Env : Context $\rightarrow$ Set where

ε : Env ε

$_._$: Env $\Gamma \rightarrow$ Value $a \rightarrow$ Env ($\Gamma . a$)

lookup : Env $\Gamma \rightarrow a \in \Gamma \rightarrow$ Value a

lookup ($\rho . v$) zero = v

lookup ($\rho . v$) (suc x) = lookup $\rho\ x$

eval : Env $\Gamma \rightarrow \Gamma \vdash a \rightarrow$ Value a

eval ρ (var x) = lookup $\rho\ x$

eval ρ (abs t) = $\lambda v \rightarrow$ eval ($\rho . v$) t

eval ρ (app $t\ u$) = eval $\rho\ t$ (eval $\rho\ u$)

What this already proves

`eval` is a total function accepted by Agda, so the model is a genuine one. Interpreting every atom by the empty type:

```
module NoAtom where
  open Den ( $\lambda \_ \rightarrow \perp$ )

  noAtom :  $\varepsilon \vdash ' \alpha \rightarrow \perp$ 
  noAtom t = eval  $\varepsilon$  t
```

No closed derivation of an atomic type — the counterpart of *no atom is derivable* from Lecture 6, obtained from the same one-line argument: apply soundness to the derivation.

Peirce's formula is not refuted this way. As in Lecture 6, a finer notion of model will be needed.

Peirce's formula resists induction

The question

Peirce : Ty

Peirce = $((X \Rightarrow Y) \Rightarrow X) \Rightarrow X$

Lecture 6 showed that Peirce's law is not derivable in the Hilbert system. Here the question is about natural deduction: is there a closed derivation, that is, an element of $\varepsilon \vdash \text{Peirce}$?

Let us try what always worked in Lectures 5–7: induction on the derivation.

A naive induction

```
-- noPeirce :  $\varepsilon \vdash \text{Peirce} \rightarrow \perp$ 
-- noPeirce (var ())
-- noPeirce (abs (var (suc ())))
-- noPeirce (abs (app t u)) = ?
-- noPeirce (app t u)      = ?
```

The first two clauses are fine: a variable in ε does not exist, and the only assumption does not have type 'X.

The two remaining clauses are stuck. In `app t u` the function part has type $b \Rightarrow \text{'X}$ for a completely **unconstrained** b . Case analysis on t produces another application, with another unconstrained type. The induction does not decrease anything.

Diagnosis

The elimination rule is applicable at every type, and its premise mentions a type that does not occur in its conclusion. Three observations restrict it:

1. To prove an implication it is sufficient to use λ -abstraction.
2. The elimination rule should not be applied to an implication that came from a λ -abstraction: that is a detour, removed by β (local soundness, Lecture 3).
3. A chain of applications should only ever be used to consume an *assumption*.

Each observation cuts away derivations without changing what is derivable. Together they define **normal** derivations.

Normal and neutral derivations

Two judgments, defined together

Neutral derivations $\Gamma \vdash_u a$: an assumption consumed by a chain of applications. Never the head of a β -redex.

$$\frac{x:a \in \Gamma}{\Gamma \vdash_u x : a} \qquad \frac{\Gamma \vdash_u t : a \Rightarrow b \quad \Gamma \vdash_n u : a}{\Gamma \vdash_u t u : b}$$

Normal derivations $\Gamma \vdash_n a$: abstractions, and neutrals.

$$\frac{\Gamma \vdash_u t : a}{\Gamma \vdash_n t : a} \qquad \frac{\Gamma, x:a \vdash_n t : b}{\Gamma \vdash_n \lambda x. t : a \Rightarrow b}$$

The application rule has moved: its function premise is now *neutral*, so observation 2 holds by construction.

Normal forms in Agda

mutual

infix 4 $_ \vdash_u _ _ \vdash_n _$

data $_ \vdash_u _$ ($\Gamma : \text{Context}$) : ($a : \text{Ty}$) $\rightarrow$ Set where

var : $a \in \Gamma \rightarrow \Gamma \vdash_u a$

app : $\Gamma \vdash_u a \Rightarrow b \rightarrow \Gamma \vdash_n a \rightarrow \Gamma \vdash_u b$

data $_ \vdash_n _$ ($\Gamma : \text{Context}$) : ($a : \text{Ty}$) $\rightarrow$ Set where

ne : $\Gamma \vdash_u a \rightarrow \Gamma \vdash_n a$

abs : $\Gamma \cdot a \vdash_n b \rightarrow \Gamma \vdash_n a \Rightarrow b$

Two mutually defined families: **mutual** because $_ \vdash_u _$ mentions $_ \vdash_n _$ in its argument position, and $_ \vdash_n _$ mentions $_ \vdash_u _$ in its conclusion.

Lemma 1: no closed neutral derivations

$\text{noClosedNeutral} : \varepsilon \vdash_u a \rightarrow \perp$
 $\text{noClosedNeutral} (\text{var } ())$
 $\text{noClosedNeutral} (\text{app } u \ n) = \text{noClosedNeutral } u$

A neutral derivation has an assumption at its head; in the empty context there is none.

This induction *does* terminate: the recursive call is on the function part, and the chain of applications is finite. Nothing is claimed about types.

Now for the interesting half.

Lemma 2, first attempt

Goal: $X \Rightarrow Y$ is not derivable from $(X \Rightarrow Y) \Rightarrow X$. (Otherwise there would be a proof of Peirce.)

```
-- lemma :  $\varepsilon \cdot (\neg X \Rightarrow \neg Y) \Rightarrow \neg X \vdash_n \neg X \Rightarrow \neg Y \rightarrow \perp$ 
-- lemma (ne (var (suc ())))
-- lemma (abs (ne (var (suc (suc ())))))
-- lemma (ne (app u n)) = ?
-- lemma (abs (ne (app u n))) = ?
```

The variable cases go through. The application cases do not: in `app u n` we see the *last* argument first, while the information we need — which assumption is at the head — sits arbitrarily deep inside u .

Applications are nested the wrong way round.

Spines

Present a chain of applications as a head plus a list of arguments. A **spine** $\Gamma \mid a \vdash_n c$ turns a derivation of a into one of c :

$$\frac{}{\Gamma \mid c \vdash_n c} \quad \frac{\Gamma \vdash_n a \quad \Gamma \mid b \vdash_n c}{\Gamma \mid a \Rightarrow b \vdash_n c}$$

infix 4 $_|_ \vdash_n _$

data $_|_ \vdash_n _$ ($\Gamma : \text{Context}$) : ($a \ c : \text{Ty}$) $\rightarrow$ Set where

$[] : \Gamma \mid c \vdash_n c$

$_::_ : \Gamma \vdash_n a \rightarrow \Gamma \mid b \vdash_n c \rightarrow \Gamma \mid a \Rightarrow b \vdash_n c$

$\text{snoc} : \Gamma \mid a \vdash_n b \Rightarrow c \rightarrow \Gamma \vdash_n b \rightarrow \Gamma \mid a \vdash_n c$

$\text{snoc} [] \ n = n :: []$

$\text{snoc} (m :: s) \ n = m :: \text{snoc} \ s \ n$

Neutrals in spine form

```
record NeSpine ( $\Gamma$  : Context) ( $c$  : Ty) : Set where  
  field  
    {ty} : Ty  
    head : ty  $\in$   $\Gamma$   
    spine :  $\Gamma$  | ty  $\vdash_n$  c
```

```
neSpine :  $\Gamma \vdash_u$  c  $\rightarrow$  NeSpine  $\Gamma$  c  
neSpine (var x) = record { head = x ; spine = [] }  
neSpine (app u n) with neSpine u  
... | record { head = x ; spine = s } = record { head = x ; spine = snoc s n }
```

Every neutral derivation is an assumption **head** applied to a **spine**. The type **ty** of the head is existentially quantified — an implicit **field**.

Lemma 2

```
lemma :  $\varepsilon \cdot ('X \Rightarrow 'Y) \Rightarrow 'X \vdash_n 'X \Rightarrow 'Y \rightarrow \perp$   
lemma (ne u) with neSpine u  
... | record { head = zero ; spine = t :: () }  
lemma (abs (ne u)) with neSpine u  
... | record { head = suc zero ; spine = n :: () }
```

Two clauses, and every case is absurd:

- the head can only be an assumption of the context, and there is only one (respectively, two, after `abs`);
- its type determines the spine, and the spine runs out of arguments before it reaches the target type.

The absurd pattern `()` is where the argument is made: no constructor of `_|_|_n_` fits.

Peirce has no normal derivation

$\text{noPeirce}_n : \varepsilon \vdash_n \text{Peirce} \rightarrow \perp$
 $\text{noPeirce}_n (\text{ne } u) = \text{noClosedNeutral } u$
 $\text{noPeirce}_n (\text{abs } (\text{ne } u)) \text{ with neSpine } u$
 $\dots \mid \text{record } \{ \text{head} = \text{zero} ; \text{spine} = n :: [] \} = \text{lemma } n$

A closed normal derivation of Peirce is either neutral, refuted by Lemma 1, or an abstraction. In the latter case the body derives X from the single assumption $(X \Rightarrow Y) \Rightarrow X$, so it is that assumption applied to one argument — a normal derivation of $X \Rightarrow Y$, refuted by Lemma 2.

This is a complete proof about *normal* derivations only.

Are normal forms enough?

The result so far is conditional. To conclude that Peirce is not derivable at all, we need:

for every $\Gamma \vdash a$ there is a $\Gamma \vdash_n a$.

That statement is **normalisation**. It has several proofs.

1. A syntactic one, by a well-chosen induction on types and derivations, today known as *hereditary substitution*.
2. Model-theoretic ones: interpret derivations in a structure from which a normal derivation can be read back.

We take the second route, with the models of Lecture 6.

Kripke models, recapitulated

The three ingredients

1. A preorder of **worlds** $(W, \leq)$. We read $w' \leq w$ as w' *is a future of* w (“the future is smaller”).
2. A **forcing** relation $w \Vdash_a \alpha$ on atoms, which is **monotone**: $w' \leq w$ and $w \Vdash_a \alpha$ imply $w' \Vdash_a \alpha$. Knowledge only grows.
3. Its extension to all types:

$$w \Vdash \alpha := w \Vdash_a \alpha \quad w \Vdash a \Rightarrow b := \forall w' \leq w. w' \Vdash a \rightarrow w' \Vdash b$$

The clause for $\Rightarrow$ quantifies over futures. This is what promotes an implication from a coincidence at w to a **law of consequence**.

(Lecture 6 wrote $u \leq v$ for v *is later than* u ; the direction here is the opposite, because in §8 worlds become contexts and $\leq$ becomes context extension.)

Kripke models in Agda

```
record Kripke : Set1 where
  field
    World : Set
    _≤_    : World → World → Set
    refl   : ∀ {w} → w ≤ w
    trans  : ∀ {w1 w2 w3} → w1 ≤ w2 → w2 ≤ w3 → w1 ≤ w3
    _⊨a_  : World → BaseTy → Set
    mona : ∀ {w w' α} → w ≤ w' → w' ⊨a α → w ⊨a α
```

A **record**, so a model is a *value*. Lecture 6 built one model by hand; we shall instantiate this one twice, with very different worlds, and get both refutation and normalisation from the same soundness proof.

Forcing, and its monotonicity

module Model (S : Kripke) (open Kripke S) where

private variable

w w' : World

infix 4 $_ \Vdash _$

$_ \Vdash _ : \text{World} \rightarrow \text{Ty} \rightarrow \text{Set}$

$w \Vdash (' \alpha) = w \Vdash_a \alpha$

$w \Vdash (a \Rightarrow b) = \forall \{w'\} \rightarrow w' \leq w \rightarrow w' \Vdash a \rightarrow w' \Vdash b$

$\text{mon} : \forall \{w w' a\} \rightarrow w \leq w' \rightarrow w' \Vdash a \rightarrow w \Vdash a$

$\text{mon} \{a = (' \alpha)\} \rho v = \text{mon}_a \rho v$

$\text{mon} \{a = a \Rightarrow b\} \rho f = \lambda \rho' v \rightarrow f (\text{trans } \rho' \rho) v$

$_ \Vdash _$ is a definition by recursion on the type, not an inductive family; its values are types. Monotonicity is then an induction on the type, and in the implication case it is exactly transitivity.

Two derived facts

Application must first travel to the present world:

`infixl 10 _|·._`

$_|\cdot._ : w \Vdash a \Rightarrow b \rightarrow w \Vdash a \rightarrow w \Vdash b$
 $f \Vdash \cdot v = f \text{ refl } v$

And the two Hilbert axioms of Lecture 6 are forced in every world of every model:

$\Vdash K : w \Vdash a \Rightarrow b \Rightarrow a$
 $\Vdash K \{a = a\} _ v \eta u = \text{mon } \{a = a\} \eta v$

$\Vdash S : w \Vdash (a \Rightarrow b \Rightarrow c) \Rightarrow (a \Rightarrow b) \Rightarrow a \Rightarrow c$
 $\Vdash S _ f \eta g \eta' v = f (\text{trans } \eta' \eta) v \text{ refl } (g \eta' v)$

$\Vdash K$ is where monotonicity is needed: the value received now must survive into the future.
Compare `peirce.agda` of Lecture 6.

A countermodel for Peirce

Designing the countermodel

We want a world where $(X \Rightarrow Y) \Rightarrow X$ is forced but X is not. Reason through the cases:

- If X holds now, Peirce is forced. So X must not hold now.
- If X holds nowhere, then $X \Rightarrow Y$ holds everywhere, so $(X \Rightarrow Y) \Rightarrow X$ holds nowhere and Peirce is again forced. So X must hold in some future.
- Taking Y never forced costs nothing: Peirce with $Y := \perp$ is still classically valid.

Two worlds suffice: $\text{future} \leq \text{now}$, with X forced only at future and Y nowhere.

These are the two stages of knowledge of Lecture 6, and *half* is the value given to X .

The two worlds

```
module PeirceStructure where
```

```
data World : Set where  
  now future : World
```

```
private variable
```

```
  w w' w1 w2 w3 : World
```

```
data _≤_ : (w w' : World) → Set where  
  now      : w      ≤ now  
  future   : future ≤ w
```

Every world is a future of `now`, and `future` is a future of every world. So the relation holds of three pairs — and of future $\leq$ now in *two* ways. That is why the refutation below has three cases.

A preorder, and the valuation

$\text{refl} : w \leq w$

$\text{refl} \{w = \text{now}\} = \text{now}$

$\text{refl} \{w = \text{future}\} = \text{future}$

$\text{trans} : w_1 \leq w_2 \rightarrow w_2 \leq w_3 \rightarrow w_1 \leq w_3$

$\text{trans future } _ = \text{future}$

$\text{trans now now} = \text{now}$

$\text{data } _ \Vdash_a _ : \text{World} \rightarrow \text{BaseTy} \rightarrow \text{Set where}$

$\text{futureX} : \text{future} \Vdash_a X$

$\text{mon}_a : w \leq w' \rightarrow w' \Vdash_a \alpha \rightarrow w \Vdash_a \alpha$

$\text{mon}_a \text{ future futureX} = \text{futureX}$

Reading the valuation

$_ \Vdash_a _$ has one constructor: the single fact this model knows.

Monotonicity has one clause, because the only world from which future is reachable is future itself — so the pattern `future` covers every case that can arise.

Where Lecture 6 defined forcing of atoms by recursion into $\top$ and $\perp$, here it is an inductive family. Both work; with a family, impossible cases are dispatched by the absurd pattern `()`, as the next slide shows.

Peirce is not forced now

module RefutePeirce where

open PeirceStructure

open Model record { PeirceStructure }

neverY : $\forall \{w\} \rightarrow w \Vdash_a Y \rightarrow \perp$

neverY ()

now¬X : now $\Vdash_a X \rightarrow \perp$

now¬X ()

refute : now $\Vdash$ Peirce $\rightarrow \perp$

refute f = now¬X (f now λ where

future $g \rightarrow$ futureX

(now {future}) $g \rightarrow$ futureX

(now {now}) $g \rightarrow \perp$ -elim (neverY (g future futureX)))

Reading the refutation

We must supply a proof of $\text{now} \Vdash (X \Rightarrow Y) \Rightarrow X$, that is, for every $w \leq \text{now}$, turn $w \Vdash X \Rightarrow Y$ into $w \Vdash X$. Three ways to be a future of now:

world	obligation	discharged by
future	$\text{future} \Vdash_a X$	the one fact of the model
now	$\text{now} \Vdash_a X$	the hypothesis g , which yields $\text{future} \Vdash_a Y$

In the second case X is unavailable, but so is the hypothesis: applying g in future, where X is known, would produce Y , which is known nowhere.

now forces the premise vacuously, and does not force X .

Soundness and completeness

The two directions

Much of logic consists of these two statements about a class of models.

Soundness every derivable sequent is forced in every model

Completeness every sequent forced in every model is derivable

Soundness is proved by induction on the derivation, one case per rule — as in Lecture 6. Forcing must first be extended from types to contexts.

Completeness is proved by exhibiting *one* model: if the sequent is forced in every model, it is forced in that one, and from there a derivation is extracted. Typically the model is built from syntax.

Our chosen model produces *normal* derivations. That is normalisation.

Forcing a context

```
module Soundness (S : Kripke) where

  open Kripke S
  open Model S

  private variable
    w w' : World

  infix 4 _|S_ _⊨_

  data _|S_ (w : World) : (Γ : Context) → Set where
    ε      : w |S ε
    _·_    : w |S Γ → w ⊨ a → w |S Γ · a

  monS : w ≤ w' → w' |S Γ → w |S Γ
  monS ρ ε = ε
  monS ρ (_·_ {a = a} δ v) = monS ρ δ · mon {a = a} ρ v
```

An *environment*: a value for every assumption, of the same shape as `Env` in §2.

Soundness

$\Gamma \models a$: every world forcing Γ forces a .

$_ \models _ : \text{Context} \rightarrow \text{Ty} \rightarrow \text{Set}$

$\Gamma \models a = \forall \{w\} \rightarrow w \Vdash_s \Gamma \rightarrow w \Vdash a$

$(_)_x : a \in \Gamma \rightarrow \Gamma \models a$

$(\text{zero } _)_x (\delta \cdot v) = v$

$(\text{suc } x _)_x (\delta \cdot v) = (x _)_x \delta$

$(_) : \Gamma \vdash a \rightarrow \Gamma \models a$

$(\text{var } x _)\delta = (x _)_x \delta$

$(\text{abs } t _)\delta = \lambda \eta \, v \rightarrow (t) (\text{mon}_s \eta \, \delta \cdot v)$

$(\text{app } t \, u) \delta = (t) \delta \text{ refl } ((u) \delta)$

Three clauses, one per rule. Abstraction is where the environment is transported into the future η ; application is where **refl** says *now*.

Soundness on paper

By induction on the derivation of $\Gamma \vdash a$, for arbitrary w with $w \Vdash_s \Gamma$.

- **Hypothesis** $x:a \in \Gamma$. Read the value off $w \Vdash_s \Gamma$.
- **Introduction** $\Gamma \vdash \lambda x.t : a \Rightarrow b$. Let $w' \leq w$ and $w' \Vdash a$. By monotonicity $w' \Vdash_s \Gamma$, so $w' \Vdash_s \Gamma, x:a$, and the induction hypothesis for t gives $w' \Vdash b$.
- **Elimination** $\Gamma \vdash tu : b$. The induction hypotheses give $w \Vdash a \Rightarrow b$ and $w \Vdash a$; instantiate the former at $w \leq w$, which is reflexivity.

Exactly the clauses of $(\sqsubseteq)$. The proof *is* the program.

Peirce is not derivable

```
module PeirceNotDerivable where

open RefutePeirce
open Soundness record { PeirceStructure }

peirceFalse :  $\varepsilon \vdash \text{Peirce} \rightarrow \perp$ 
peirceFalse t = refute ( $\llbracket t \rrbracket \varepsilon$ )
```

One line: evaluate the derivation in the countermodel of §6, in the empty environment, and hand the result to the refutation.

This is the first of the two proofs announced at the start. It needed a creative step — inventing the two worlds. The second proof will need none.

Validity in every model

$\text{Valid} : \text{Context} \rightarrow \text{Ty} \rightarrow \text{Set}_1$

$\text{Valid } \Gamma \ a = (S : \text{Kripke}) \rightarrow \text{Soundness.} _ \models _ \ S \ \Gamma \ a$

$\text{sound} : \Gamma \vdash a \rightarrow \text{Valid } \Gamma \ a$

$\text{sound } t \ S = \text{Soundness.} (_) \ S \ t$

Valid quantifies over *all* models, so it lives one universe up, in Set_1 — the predicativity of Lecture 4 showing up in an ordinary piece of metatheory.

sound is $(_)$ with the model turned into an argument: one proof, every model.

It remains to prove the converse. For that we must choose a model.

Building the syntactic model

The choice

Take derivations themselves as the semantics:

worlds	contexts Γ
forcing of atoms	$\Gamma \Vdash_a \alpha := \Gamma \vdash_u \alpha$, the <i>neutral</i> derivations
futures	?

Neutrals, not arbitrary derivations: a neutral derivation of an atom is already normal, and the whole point is to end up with normal derivations.

What should $\Gamma \leq \Delta$ be? The requirements are:

- it is a preorder, and
- forcing of atoms is monotone: $\Gamma \leq \Delta$ and $\Delta \vdash_u \alpha$ must give $\Gamma \vdash_u \alpha$.

Let us collect more requirements before deciding.

Reification, a first look

Soundness gives us $\Gamma \Vdash a$ from $\Gamma \vdash a$. To finish we need to read a normal derivation back out:

$$\Gamma \Vdash a \longrightarrow \Gamma \vdash_n a.$$

For an atom this is nothing: $\Gamma \Vdash \alpha$ is $\Gamma \vdash_u \alpha$. Take the smallest interesting case, $a = X \Rightarrow Y$. We are given

$$f: \forall \Delta \leq \Gamma. \Delta \vdash_u X \rightarrow \Delta \vdash_u Y$$

and want $\Gamma \vdash_n X \Rightarrow Y$. By the introduction rule it suffices to produce $\Gamma, X \vdash_n Y$.

Choose $\Delta := \Gamma, X$. Then f needs

- $\Gamma, X \leq \Gamma$ — so **a future may add an assumption**;
- $\Gamma, X \vdash_u X$ — the last variable. Available.

So $\leq$ must at least allow extension at the right end.

Weakening

Deriving the definition

The requirement was: $\Gamma \leq \Delta$ and $\Delta \vdash_u a$ give $\Gamma \vdash_u a$. Neutral and normal derivations are defined together, so this must be proved together with $\Gamma \leq \Delta$ and $\Delta \vdash_n a$ giving $\Gamma \vdash_n a$.

The abstraction case of the second statement has premise $\Delta, a \vdash_n b$, so the induction hypothesis is needed at $\Gamma, a \leq \Delta, a$. Collecting:

$\varepsilon \leq \varepsilon$	the base case
$\Gamma \leq \Delta$ implies $\Gamma, a \leq \Delta$	extension, needed for reification
$\Gamma \leq \Delta$ implies $\Gamma, a \leq \Delta, a$	needed under a binder

Read these three as constructors: that is the definition. Nothing more is needed, so take nothing more.

Weakenings

infix 4 $_ \leq _$

data $_ \leq _$: ($\Gamma \Delta$: Context) $\rightarrow$ Set where

done : $\varepsilon \leq \varepsilon$

skip : $\Gamma \leq \Delta \rightarrow \Gamma \cdot a \leq \Delta$

keep : $\Gamma \leq \Delta \rightarrow \Gamma \cdot a \leq \Delta \cdot a$

$\Gamma \leq \Delta$ says: Δ is obtained from Γ by deleting assumptions; equivalently, Γ is Δ with assumptions inserted anywhere. The constructor names describe how Δ is traversed.

These are **order-preserving embeddings**, or proof-relevant subsequences: not a proposition but data, since the same Δ may sit inside Γ in several ways.

Weakenings form a category

$\text{idW} : \Gamma \leq \Gamma$

$\text{idW} \{ \Gamma = \varepsilon \} = \text{done}$

$\text{idW} \{ \Gamma = \Gamma \cdot a \} = \text{keep idW}$

$\text{infixl } 8 \text{ } _._$

$_._ : \Gamma \leq \Delta \rightarrow \Delta \leq \Phi \rightarrow \Gamma \leq \Phi$

$\text{done} \cdot \rho' = \rho'$

$\text{skip } \rho \cdot \rho' = \text{skip } (\rho \cdot \rho')$

$\text{keep } \rho \cdot \text{skip } \rho' = \text{skip } (\rho \cdot \rho')$

$\text{keep } \rho \cdot \text{keep } \rho' = \text{keep } (\rho \cdot \rho')$

Composition is written in diagrammatic order. Identity and associativity hold, and their proofs are in `Term/Weakening/Properties.agda`; we do not need them today, which is why forcing was defined by recursion rather than as a quotient.

Weakening acts

$$\text{weak}_x : \Gamma \leq \Delta \rightarrow a \in \Delta \rightarrow a \in \Gamma$$

$$\text{weak}_x (\text{keep } \rho) \text{ zero} = \text{zero}$$

$$\text{weak}_x (\text{keep } \rho) (\text{suc } x) = \text{suc } (\text{weak}_x \rho x)$$

$$\text{weak}_x (\text{skip } \rho) x = \text{suc } (\text{weak}_x \rho x)$$

$$\text{weak} : \Gamma \leq \Delta \rightarrow \Delta \vdash a \rightarrow \Gamma \vdash a$$

$$\text{weak } \rho (\text{var } x) = \text{var } (\text{weak}_x \rho x)$$

$$\text{weak } \rho (\text{abs } t) = \text{abs } (\text{weak } (\text{keep } \rho) t)$$

$$\text{weak } \rho (\text{app } t u) = \text{app } (\text{weak } \rho t) (\text{weak } \rho u)$$

On indices, **skip** is the successor. On derivations, the action is structural and the only work happens at the variables.

...on normal and neutral derivations

mutual

$$\text{weak}_u : \Gamma \leq \Delta \rightarrow \Delta \vdash_u a \rightarrow \Gamma \vdash_u a$$

$$\text{weak}_u \rho (\text{var } x) = \text{var } (\text{weak}_x \rho x)$$

$$\text{weak}_u \rho (\text{app } u \ n) = \text{app } (\text{weak}_u \rho u) (\text{weak}_n \rho n)$$

$$\text{weak}_n : \Gamma \leq \Delta \rightarrow \Delta \vdash_n a \rightarrow \Gamma \vdash_n a$$

$$\text{weak}_n \rho (\text{ne } u) = \text{ne } (\text{weak}_u \rho u)$$

$$\text{weak}_n \rho (\text{abs } n) = \text{abs } (\text{weak}_n (\text{keep } \rho) n)$$

Weakening does not create redexes, so it preserves both judgments. This is the monotonicity our model needs.

$\Gamma \mapsto (\Gamma \vdash a)$, $\Gamma \mapsto (\Gamma \vdash_u a)$ and $\Gamma \mapsto (\Gamma \vdash_n a)$ are contravariant functors on the category of weakenings — **presheaves**.

Reification and reflection

The general case

Reification at $a \Rightarrow b$ has to turn $f: \forall \Delta \leq \Gamma. \Delta \Vdash a \rightarrow \Delta \Vdash b$ into $\Gamma \vdash_n a \Rightarrow b$. As before, take $\Delta := \Gamma, a$ and $\rho := \text{skip id} : \Gamma, a \leq \Gamma$. What is missing is an argument

$$\Gamma, a \Vdash a.$$

For atomic a that is the variable. For $a = a_1 \Rightarrow a_2$ it is not: we would have to produce a semantic *function*.

So introduce a second operation in the other direction — **reflection** of a neutral derivation into the model:

$$\Gamma \vdash_u a \longrightarrow \Gamma \Vdash a.$$

At $a_1 \Rightarrow a_2$: given $\Delta \leq \Gamma$ and $\Delta \Vdash a_1$, weaken the neutral to $\Delta \vdash_u a_1 \Rightarrow a_2$, **reify** the argument to $\Delta \vdash_n a_1$, apply, and reflect the result.

Reification and reflection are mutual, by induction on the type.

The model of neutrals

NbEStructure : Kripke

NbEStructure = record

```
{ World = Context  
;  $\_ \leq \_$  =  $\_ \leq \_$   
; refl = idW  
; trans =  $\_ \cdot \_$   
;  $\_ \Vdash_a \_$  =  $\lambda \Gamma \alpha \rightarrow \Gamma \vdash_u ' \alpha$   
; mona = weaku  
}
```

Every field was forced on us by the requirements collected in §8 and §9. There was no freedom left, which is why this model deserves to be called *generic*.

Reify and reflect

module NbE where

open Model NbEStructure

open Soundness NbEStructure

mutual

reify : $\Gamma \Vdash a \rightarrow \Gamma \vdash_n a$

reify $\{a = ' \alpha\}$ $u = \text{ne } u$

reify $\{a = a \Rightarrow b\}$ $f =$

$\text{abs } (\text{reify } (f (\text{skip idW}) (\text{reflect } \{a = a\} (\text{var zero}))))$

reflect : $\Gamma \vdash_u a \rightarrow \Gamma \Vdash a$

reflect $\{a = ' \alpha\}$ $u = u$

reflect $\{a = a \Rightarrow b\}$ $u \rho v = \text{reflect } (\text{app } (\text{weak}_u \rho u) (\text{reify } v))$

Both recurse on the type, and each calls the other at a smaller type: **reify** at $a \Rightarrow b$ calls **reflect** at a , and **reflect** at $a \Rightarrow b$ calls **reify** at a .

The identity environment

Soundness needs an environment. In this model there is a canonical one: reflect every assumption.

$$\Vdash_{\text{id}} : \Gamma \Vdash_s \Gamma$$

$$\Vdash_{\text{id}} \{\Gamma = \varepsilon\} = \varepsilon$$

$$\Vdash_{\text{id}} \{\Gamma = \Gamma \cdot a\} = \text{mon}_s (\text{skip idW}) \Vdash_{\text{id}} \cdot \text{reflect} \{a = a\} (\text{var zero})$$

The induction is on the context. The recursive call produces $\Gamma \Vdash_s \Gamma$, which must be transported to the larger world $\Gamma \cdot a$ — this is where monotonicity of the environment is used.

Completeness

`complete` : `Valid` $\Gamma \ a \rightarrow \Gamma \vdash_n a$
`complete` $f = \text{reify } (f \text{ NbEStructure } \Vdash_{\text{id}})$

A sequent forced in every model is forced in `NbEStructure`; instantiate it at the identity environment and reify.

Completeness with respect to Kripke models, with *normal* derivations as the conclusion. The proof is three function calls.

Compare the classical situation: completeness for Boolean models needs classical reasoning and yields no algorithm. Here the model is constructive and the proof is a program.

Normalisation

$\text{norm} : \Gamma \vdash a \rightarrow \Gamma \vdash_n a$
 $\text{norm } t = \text{complete } (\text{sound } t)$

$\text{norm}' : \Gamma \vdash a \rightarrow \Gamma \vdash_n a$
 $\text{norm}' t = \text{reify } (\llbracket t \rrbracket \Vdash_{\text{id}})$

norm is soundness followed by completeness. norm' is the same function with the detour through all models removed: **evaluate, then reify**.

This is **normalisation by evaluation**. Compare eval of §2: the same evaluator, run in a different model.

Peirce, second proof

Composing `norm` with the refutation of §4:

$$\text{noPeirce}_n \circ \text{NbE.norm} : \varepsilon \vdash \text{Peirce} \rightarrow \perp$$

First proof (§7) a countermodel invented for Peirce

Second proof normalisation, then a finite inspection of normal forms

The second proof does more work but requires no invention: the model is determined, and the case analysis on normal derivations is mechanical — the kind of search that fails on arbitrary derivations but terminates on normal ones.

Summary

What was built

Judgment	Agda	Role
$\Gamma \vdash a$	$_ \vdash _$	derivations
$\Gamma \vdash_u a$	$_ \vdash_u _$	neutral: assumption applied
$\Gamma \vdash_n a$	$_ \vdash_n _$	normal: no detours
$\Gamma \mid a \vdash_n c$	$_ \mid _ \vdash_n _$	spine, head made accessible
$\Gamma \leq \Delta$	$_ \leq _$	weakening, and the futures
$w \Vdash a$	$_ \Vdash _$	forcing, by recursion on a
$\Gamma \models a$	$_ \models _$	validity in one model
	Valid	validity in every model
$(_ _)$		soundness, by induction on derivations
reify / reflect		by induction on types
norm		their composition

Lessons

- Both refutations of Peirce go through a Kripke model: once by a creative construction of a countermodel, once through normalisation, which is a model too.
- Normalisation for STLC can also be had by an elementary syntactic induction. For richer systems that route becomes hard, and models remain available.
- A negative result about a calculus is a positive statement about a model. The finer the notion of model, the more it refutes — truth tables, then two stages of knowledge, then contexts and neutrals.
- Restricting the rules (normal derivations) turns an unbounded search into a finite one. Making the restriction harmless is the theorem.

Loose ends

- We proved only that a normal derivation *exists*. With an operational semantics one wants the normal form to be a *reduct* of the original derivation.
- `norm` does in fact produce one — anything else would be hard to arrange — but our development gives no evidence of it. The model relates worlds to types, not to terms.
- To obtain the evidence, the semantic values must be indexed by the term they interpret: $\Gamma \Vdash a$ becomes a relation between derivations and values. That is the method of *reducibility*, and it also yields decidability of $\beta\eta$ -equality.
- Carried out in `Term/Normalization/EtaLong.agda` and `Term/Equality.agda` in the course sources.

Exercises

1. Add products to the calculus: extend `Ty`, `_|_`, the normal forms, and the clause of `_|_`. Which of `reify`, `reflect`, `(|_)` change?
2. Show that the countermodel of §6 also refutes $\neg\neg X \Rightarrow X$, with $\neg a := a \Rightarrow Y$ and Y never forced.
3. Prove `weakx idW` $x \equiv x$ and the corresponding statement for `weak`. Where would these be needed?
4. Instantiate `norm` at a Church numeral and read off the result. Compare with `Den.eval`.
5. Why does `_|_` quantify over futures in the clause for $\Rightarrow$? Drop the quantification and find the first proof that breaks.

References

- Saul Kripke. Semantical analysis of intuitionistic logic I. In *Formal Systems and Recursive Functions*, 1965.
- Anne S. Troelstra, Dirk van Dalen. *Constructivism in Mathematics*. North-Holland, 1988. (Kripke models, forcing)
- John C. Mitchell, Eugenio Moggi. Kripke-style models for typed λ -calculus. *Ann. Pure Appl. Logic* 51, 1991.
- Ulrich Berger, Helmut Schwichtenberg. An inverse of the evaluation functional for typed λ -calculus. *LICS*, 1991. (normalisation by evaluation)
- Catarina Coquand. From semantics to rules: a machine assisted analysis. *CSL*, 1993. (the proof formalised here)
- Thorsten Altenkirch, Martin Hofmann, Thomas Streicher. Categorical reconstruction of a reduction free normalization proof. *CTCS*, 1995.
- Frank Pfenning, Rowan Davies. A judgmental reconstruction of modal logic. *MSCS* 11(4), 2001. (verifications and uses)
- Kevin Watkins, Iliano Cervesato, Frank Pfenning, David Walker. A concurrent logical framework I. CMU, 2002. (hereditary substitution)
- Chantal Keller, Thorsten Altenkirch. Hereditary substitutions for simple types, formalized. *MSFP*, 2010.
- Andreas Abel. *Normalization by Evaluation: Dependent Types and Impredicativity*. Habilitation thesis, LMU Munich, 2013.